

Proyecto 2

1 Descripción del proyecto

El proyecto es individual. Consiste en diseñar y desarrollar una aplicación web completa para gestionar el inventario y las ventas de una tienda. La aplicación debe incluir un frontend en React, una API REST en el backend, y debe desplegarse mediante Docker.

El dominio de negocio es el mismo para todos: la tienda maneja productos agrupados en categorías, comprados a proveedores. Los clientes realizan compras atendidas por empleados, y cada compra puede incluir varios productos. El enfoque del proyecto está en la arquitectura, el diseño de la interfaz y la calidad del desarrollo web; la base de datos se provee como punto de partida (esquema y datos de prueba incluidos en el repositorio del curso).

El puntaje total posible supera los 100 puntos, pero la nota máxima acreditable es 100 y para obtener una nota distinta de 0 el proyecto debe estar subido a un servidor en producción.

2 Restricciones técnicas

- **Frontend obligatorio en React.** El stack del backend es libre.
- **API REST obligatoria.** Toda comunicación entre frontend y backend debe realizarse mediante una API REST con respuestas en JSON. No se aceptan aplicaciones monolíticas con renderizado en servidor.
- **Docker obligatorio.** Toda la infraestructura (base de datos, backend, frontend) debe definirse en un `docker-compose.yml`. El proyecto debe levantarse únicamente con `docker compose up`.
- **Variables de entorno:** todas las credenciales deben gestionarse mediante un archivo `.env` (no hardcodeadas). El repositorio debe incluir un `.env.example` con las variables requeridas.
- **Credenciales fijas para calificación:** el usuario de base de datos debe ser `proy2` y la contraseña `secret`. No se calificará un proyecto que use credenciales distintas.
- Todo el trabajo se entrega en un repositorio de GitHub. El README debe permitir levantar el proyecto desde cero.

3 Rúbrica de evaluación

Cada tarea se califica como aprobada o no aprobada; no hay puntaje parcial por tarea. Se suman los puntos de las tareas aprobadas. La nota máxima acreditable es **100 puntos**.

I. Arquitectura y API REST

Endpoints REST documentados (README o colección Postman/OpenAPI incluida)	8
CRUD completo expuesto vía API para al menos 2 entidades (productos, ventas, clientes, etc.)	15
Manejo de errores en la API: códigos HTTP correctos y mensajes de error en JSON	7
Al menos 1 endpoint que agregue datos (total de ventas, stock disponible, etc.)	5
Subtotal posible	35

II. Frontend — React

Todos los criterios de esta categoría deben ser verificables directamente en la interfaz de la aplicación.

Navegación entre vistas implementada con React Router (mínimo 4 rutas distintas)	8
Estado global manejado con React Context (sesión de usuario, carrito u otro flujo transversal)	8
Uso correcto de hooks: useState, useEffect, y al menos uno de useCallback o useMemo	8
Al menos 1 flujo de estado complejo manejado con useReducer (carrito, filtros combinados, etc.)	8
Formularios controlados con validación del lado del cliente para las operaciones CRUD	8
Al menos 1 reporte visible en la UI con datos reales (tablas, gráficas o resúmenes)	8
Manejo visible de errores para el usuario (mensajes de validación, feedback de operaciones)	5
Subtotal posible	53

III. Calidad de código

Configuración de linter funcional (ESLint) sin errores al ejecutar el comando de lint	5
Al menos 3 pruebas unitarias o de integración que pasen (vitest, jest o equivalente)	7
Subtotal posible	12

IV. Despliegue y entrega

README con instrucciones funcionales y ejemplo de <code>docker compose up</code>	5
El proyecto levanta correctamente con <code>docker compose up</code> sin pasos adicionales	10
Subtotal posible 15	

V. Avanzado

Autenticación de usuarios con login/logout y estado de sesión manejado mediante Context	10
Exportar al menos 1 reporte a CSV o PDF desde la UI	5
Diseño responsivo verificable en móvil y escritorio	5
Subtotal posible 20	

Total posible 135

Nota máxima acreditable 100

Notas:

- Un proyecto que no levante con `docker compose up` siguiendo el README no se evalúa en las categorías II–V.
- Un proyecto con credenciales distintas a `proy2 / secret` no se evalúa en ningún criterio que requiera conexión a la base de datos.
- La comunicación frontend–backend debe ser exclusivamente mediante la API REST. El acceso directo a la base de datos desde el frontend invalida la categoría I.
- Las entregas se realizan según las fechas publicadas en Canvas. Todo el trabajo debe estar en el repositorio de GitHub antes del cierre.